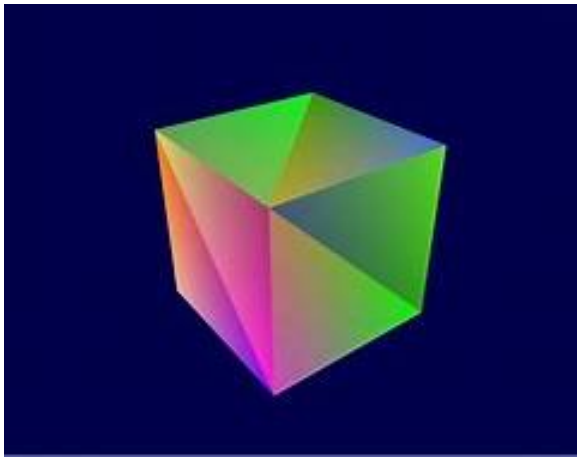
Exercice 1 (obligatoire) : Affichage 3D perspective.

Affichez le cube 3D suivant en perspective et en rotation. Toutes les matrices sont des matrices homogènes 4x4.

```
const GLfloat cube_vertices[] = {
// front
-1.0, -1.0,  1.0,   1.0, -1.0,  1.0,   1.0,  1.0,  1.0,  -1.0,  1.0,  1.0,
// back
-1.0, -1.0, -1.0,   1.0, -1.0, -1.0,   1.0,  1.0, -1.0,  -1.0,  1.0, -1.0
};
```



```
const GLushort cube_elements[] = {
// front
0,1,2, 2,3,0,
// right
1,5,6, 6,2,1,
// back
7,6,5, 5,4,7,
// left
4,0,3, 3,7,4,
// bottom
4,5,1, 1,0,4,
// top
3,2,6, 6,7,3
};
```

Vous pouvez ajouter un attribut de couleur aux sommets ou convertir les positions en couleur dans le shader.

Les **rotations** doivent suivre **l'ordre** suivant : autour de l'axe Up, puis autour de l'axe Right, puis autour de l'axe Forward.

https://fr.wikipedia.org/wiki/Matrice_de_rotation

Attention, par défaut le cube centré sur son origine en local qui est (0,0,0). Ce qui signifie que dans le monde le cube est donc positionné au même endroit que la caméra. Veillez donc à bien **translater** le cube dans la scène (rappel : le repère d'OpenGL est un repère main droite dont l'axe "forward" pointe HORS de l'écran).

Rappel: l'ordre des matrices est le suivant : Projection <- Camera <- Monde <- position

Il n'est pas demandé de gérer une caméra à ce stade.

Exercice 1.2 (optionnel) : Affichage 2D en pixel

Affichez des formes simples en coordonnées pixels 2D (et pas NDC) **PAR-DESSUS** le rendu 3D.

- Définir une matrice de projection orthographique exprimant le repère 2D en pixels dont l'origine (0,0) est en haut à gauche.
- Spécifiez des coordonnées en pixels à vos primitives 2D.
- Pensez à désactiver le Culling et le Depth Test (voir la page suivante)

Rappel : Tri des faces

A ce stade le rendu est presque correct. Le seul problème est que le rendu est parfois un peu bizarre. C'est dû au fait que les faces sont dessinées dans un ordre arbitraire. Il arrive donc que certaines parties de faces qui devraient être masquées apparaissent en avant plan car elles sont dessinées en dernier.

La solution en apparence est simple, il faut trier les faces, mais comme il peut y'avoir des intersections il faudrait découper les faces avant le tri, à chaque frame, afin de s'assurer que les portions de faces masquées ne passent pas en avant plan.

C'est complexe, mais heureusement il y'a une solution plus simple qui est l'algorithme du **Z-Buffer**. Les GPU implémentent cet algorithme en interne sous la forme d'un **Depth Buffer**, une zone mémoire stockant la profondeur de chaque pixel (la valeur stockée est en fait 1.0 / Z pour une raison qui sera vue en cours).

En 3D il est donc souvent impératif d'allouer un Depth Buffer et d'activer le test de profondeur (Depth Test, qui est désactivé par défaut en OpenGL). GLFW créé toujours un Depth Buffer par défaut dans le Framebuffer, il nous suffit simplement d'activer le Depth Test ainsi :

glEnable(**GL_DEPTH_TEST**);

On veillera à appeler **glDisable**(**GL_DEPTH_TEST**) lorsque l'on n'a plus besoin de trier les pixels (en 2D par exemple).

Par défaut, le test de profondeur fonctionne sur le mode "inférieur strict" (**GL_LESS**) c'est à dire qu'un fragment ne sera affiché à l'écran qui si son 'Z' est strictement inférieur à celui déjà présent dans le Z-Buffer. Il est possible de changer le type de test à l'aide de la fonction glDepthFunc(), exemple **glDepthFunc**(**GL_LEQUAL**) qui autorise alors le rendu d'un fragment dont le Z est égal à celui déjà présent dans le Depth Buffer.

Note : le 'Z' pris en compte est la composante Z normalisée en NDC mais convertie entre [0;1] plutôt qu'entre [-1;1]. C'est ce qu'OpenGL appelle les "Depth Bounds".

Mais il manque un détail très important : le Depth Buffer est un "buffer" comme son nom l'indique. Ce qui signifie que le contenu persiste tant qu'il n'est pas réinitialisé (ou effacé dans notre cas).

L'erreur la plus courante après avoir activé le "Depth Test" est l'absence d'affichage à l'écran. Ceci s'explique généralement par le fait que la première fois, le rendu s'effectue correctement, mais la seconde fois, la géométrie est de nouveau testée et comme le Z de tous les fragments n'est pas strictement inférieur au Z stocké dans le Depth Buffer (ils sont égaux) rien ne s'affiche.

<u>Très important :</u> On veillera donc à ajouter à **glClear**() le masque suivant **GL_DEPTH_BUFFER_BIT** en plus de **GL_COLOR_BUFFER_BIT**.

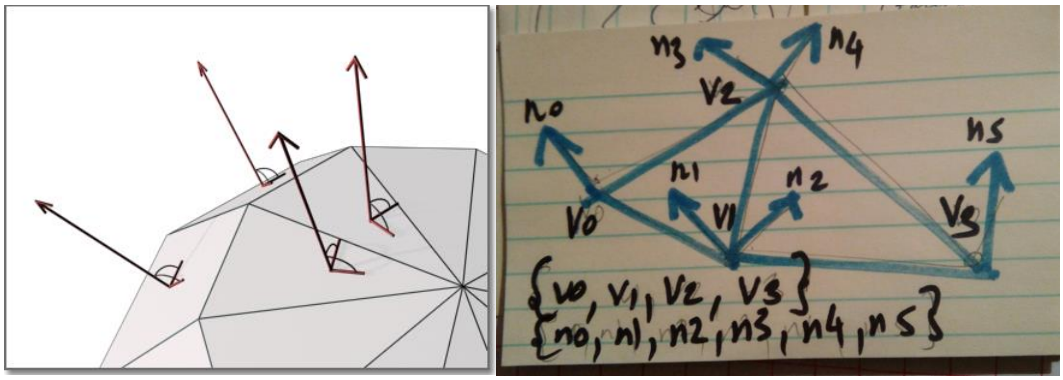
Exercice 1.2 (obligatoire) : activer le Depth Test et le Culling

N'oubliez pas qu'en 3D il est souvent inutile d'afficher les faces cachées, pensez donc à activer la <u>suppression des faces arrière</u>.

Exercice 2.1 (recommandé, optionnel) : génération de normales

Le cube que l'on a utilisé précédemment est un peu simpliste dans le sens où un sommet est toujours unique. Or il arrive fréquemment qu'un même sommet (essentiellement une position 3D ici) soit dupliqué car des faces partageant le même sommet peuvent avoir des attributs différents. Typiquement cela peut être le cas de n'importe quel attribut en dehors de la position du sommet.

Avec la forme actuelle il n'est par exemple pas possible de représenter des normales différentes à chaque face. Lorsque l'on souhaite avoir des normales aux faces (aussi appelées "hard edges", arêtes rigides) il est nécessaire que les 3 sommets d'une même face disposent de la même normale (n'oubliez pas que le rasterizer interpole tout ce qui est passé en sortie du Vertex Shader).



On a donc plus de normales que de positions. Comme OpenGL ne peut pas gérer des tableaux d'attributs qui ont un nombre différent de valeur par sommet, on est alors obligé de dédoubler (voire décupler) les positions à chaque normale unique. Dans l'exemple on doit donc dédoubler v1 avec n1 et n2, de même pour v2 avec n3 et n4.

Dans le cas de notre cube, on a 12 triangles (2 x 6 faces) ce qui nous donne nécessairement 24 sommets, donc 24 positions et 24 normales.

```
const GLfloat g_cube_vertices2[] = {
-1.0f,-1.0f,-1.0f, -1.0f,-1.0f, 1.0f, -1.0f, 1.0f, 1.0f, // Left Side -1.0f,-
1.0f,-1.0f, -1.0f, 1.0f, 1.0f, -1.0f, 1.0f,-1.0f, // Left Side
1.0f, 1.0f,-1.0f, -1.0f,-1.0f,-1.0f, -1.0f, 1.0f,-1.0f, // Back Side
1.0f,-1.0f, 1.0f, -1.0f,-1.0f,-1.0f, 1.0f,-1.0f,-1.0f, // Bottom Side
1.0f, 1.0f,-1.0f, 1.0f,-1.0f,-1.0f, -1.0f,-1.0f,-1.0f, // Back Side
1.0f,-1.0f, 1.0f, -1.0f,-1.0f, 1.0f, -1.0f,-1.0f,-1.0f, // Bottom Side
-1.0f, 1.0f, 1.0f, -1.0f,-1.0f, 1.0f, 1.0f,-1.0f, 1.0f, // Front Side
1.0f, 1.0f, 1.0f, 1.0f,-1.0f,-1.0f, 1.0f, 1.0f,-1.0f, // Right Side 1.0f,-1.0f,-
1.0f, 1.0f, 1.0f, 1.0f, 1.0f,-1.0f, 1.0f, // Right Side
1.0f, 1.0f, 1.0f, 1.0f, 1.0f,-1.0f, -1.0f, 1.0f,-1.0f, // Top Side
1.0f, 1.0f, 1.0f, -1.0f, 1.0f,-1.0f, -1.0f, 1.0f, 1.0f, // Top Side
1.0f, 1.0f, 1.0f, -1.0f, 1.0f, 1.0f, 1.0f,-1.0f, 1.0f // Front Side };
```

L'exercice consiste ici à calculer, en C++, les normales de chaque triangle à l'aide du **produit vectoriel** puis affecter cette normale à chaque sommet du triangle.

Note : il n'est pas nécessaire d'utiliser un index buffer ici.

Pour tester votre code, je vous propose le **Fragment Shader** suivant qui implémente la loi du cosinus de Lambert, ou illumination de Lambert, que nous verrons en détail dans le cours sur l'illumination :

```
varying vec3 v_Normal;
const vec3 LightDirection = vec3(1.0, -1.0, -1.0);

void main() {
  vec3 N = normalize(v_Normal); // normale en repere monde
  vec3 L = normalize(-LightDirection); // direction VERS la lumiere

  float LambertDiffuse = max(dot(N, L), 0.0);
  gl_FragColor = vec4(vec3(LambertDiffuse), 1.0);
}
```

Note : v_Normal est le résultat de la multiplication de la normale du sommet par la matrice monde spécifique aux normales. En gros, il s'agit de la matrice monde mais sans la translation et sans scale / deformation.

Exercice 2.2 (optionnel) : arêtes adoucies

Les normales aux faces (hard edges) fonctionnent bien avec les modèles architecturaux (ou même un cube) car elles mettent en valeur les échancrures du modèle.

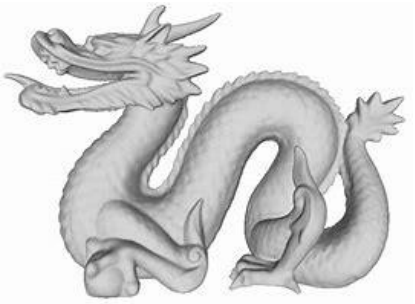
Dans le cas où l'on souhaite avoir un rendu plus adouci il faut utiliser des "soft edges" (aussi appelés "smoothing groups"). Le choix entre "hard edges" ou "soft edges" est généralement spécifié par l'artiste (globalement, ou par groupe de triangles) mais il est possible de calculer une normale au vertex qui serait la moyenne des normales aux faces dont le sommet fait partie.

Voir http://www.bytehazard.com/articles/vertnorm.html pour les différentes méthodes.

Exercice 3 (à rendre) : Objet 3D, le Stanford Dragon

Affichez le cube et le Stanford Dragon en même temps.

Le fichier dragondata.h défini deux tableaux :



- DragonVertices[133140], un tableau de float-s

- DragonIndices[45000], un tableau d'indices en unsigned short (uint16_t).

Les sommets du dragon sont structurés ainsi : `{[X,Y,Z], [NX,NY,NZ], [U,V]}` soit 8 floats par sommets. Les 3 premiers réels correspondent à la position 3D `(X,Y,Z)`, les 3 suivants à la normale "soft" `(NX,NY,NZ)`, normale du sommet (*"Vertex Normal"*). La normale est déjà *smooth* donc il n'est pas utile de procéder comme dans l'exercice 2.

Les 2 derniers réels sont des coordonnées de texture qui nous servirons pour la prochaine séance.

- Positionnez le dragon dans l'espace de sorte que l'on puisse le voir entièrement en train de tourner autour de son axe Up (Y)
- Appliquez le Fragment Shader de l'exercice 2 afin de montrer que les normales fonctionnent.